

Beyond the Binary Barrier

Extending RetroWrite with memcpy-safe sanitisation, standalone coverage, and native function tracing.

GROUP

Chirag Suthar · 2025MCS2105

Haleel Sada · 2025MCS2741

REFERENCE

Dinesh et al., RetroWrite,
IEEE S&P 2020.

Most bugs are memory bugs.

70%

of CVEs are memory-safety issues.

Source: Szekeres et al., 2013.

01

Closed source

Browsers, firmware, games ship as stripped ELF / PE binaries.

02

Dynamic tools are slow

QEMU 10–100×, Valgrind up to 300× — too slow for real fuzzing.

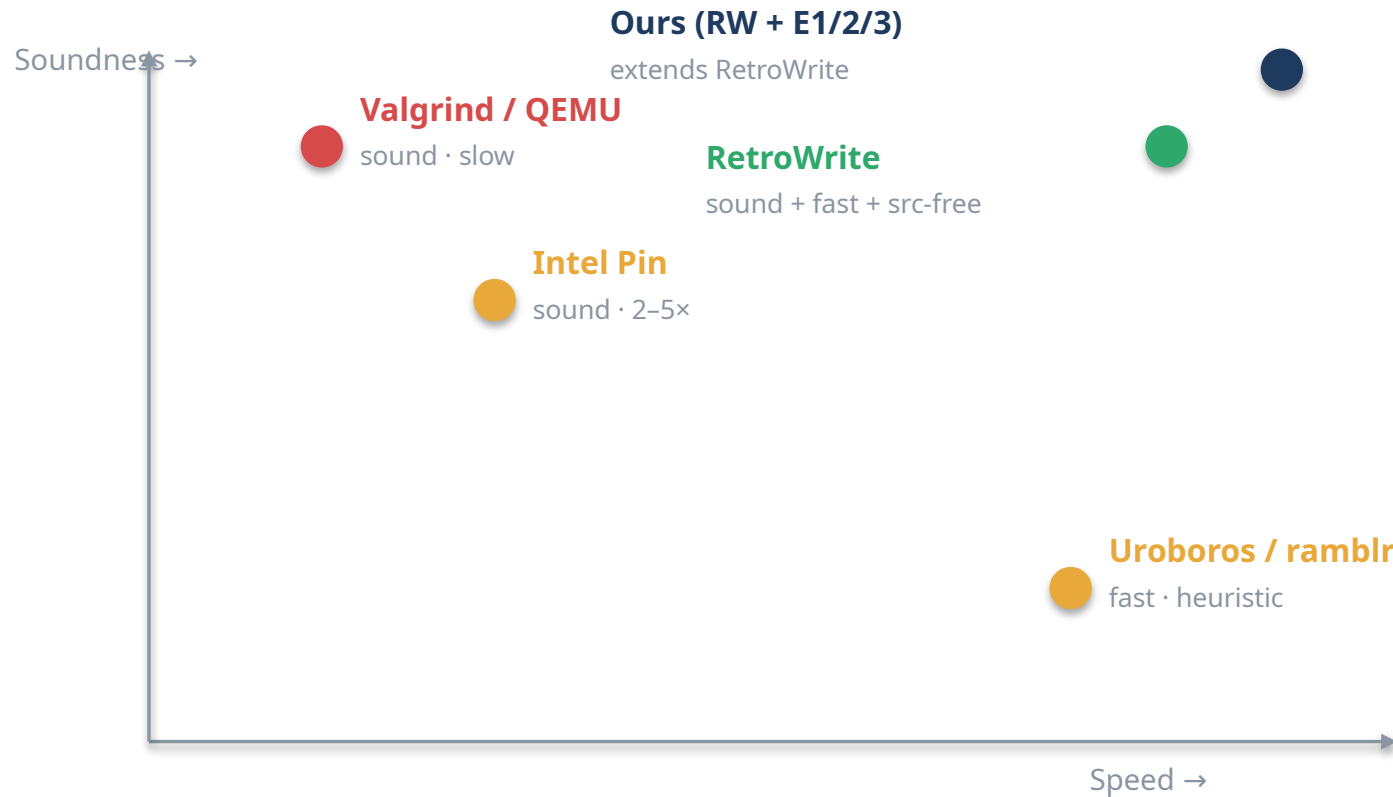
03

Static rewriters guess

Uroboros, ramblr rely on heuristics that break on real binaries.

We close three gaps in RetroWrite — memcpy sanitisation, x64 coverage, and a native tracer.

The classical trilemma.

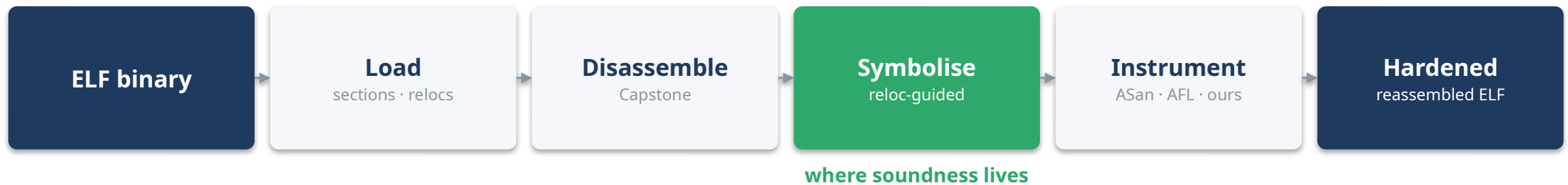


THE INSIGHT

**Relocations in PIE ELF's
label which bytes are
addresses — exactly.**

No guessing. Fast and sound at once. Our
passes inherit this guarantee.

How binary rewriting works.



PRIOR STATIC REWRITERS

Guess: is 0x401050 an address or a constant?

Heuristics break on real ELF's — reassembled binaries crash.

RETROWRITE

.rela.dyn is a ground-truth oracle.

Every address-valued immediate is labelled — symbolisation is exact.

ASan and AFL, in a sentence.

ADDRESS SANITIZER

Shadow memory catches out-of-bounds and use-after-free.

- One shadow byte per 8 application bytes
- Every load/store calls `__asan_load1/4/8`
- Red-zones around heap and stack allocations

AMERICAN FUZZY LOP

Edge-hash coverage guides input mutation.

- 64 KiB bitmap indexed by `cur ⊕ prev`
- Small stub at every basic-block entry
- New bitmap bit = new test case saved

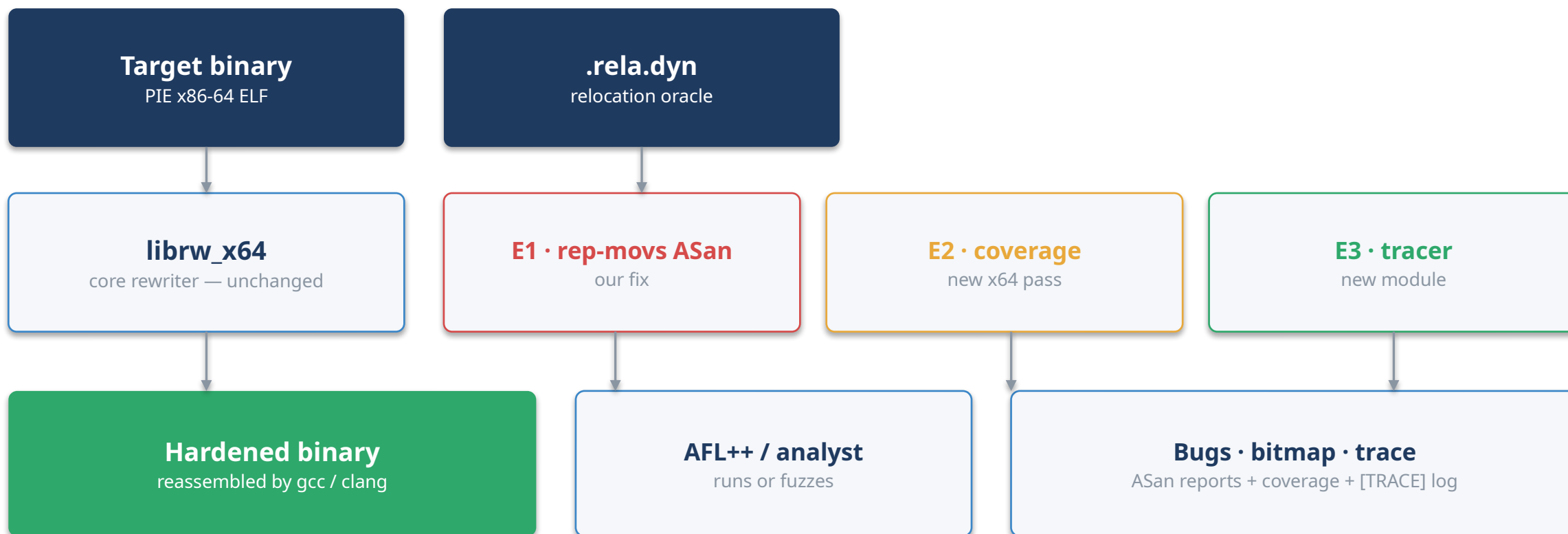
Both normally need source. RetroWrite brings them to stripped binaries.

Related work.

Tool	Sound	Fast	Src-free	Notes
QEMU	yes	10–100×	yes	too slow for fuzzing
Valgrind	yes	20–300×	yes	heavyweight
Intel Pin	yes	2–5×	yes	proprietary
Uroboros / ramblr	part.	fast	yes	heuristic symbolisation
DynInst	part.	fast	yes	unsafe on stripped ELF
E9Patch	yes	fast	yes	patch-style, limited semantics
RetroWrite	yes	fast	yes	no rep-movs · no x64 cov · no tracer
Ours (RW + E1/2/3)	yes	fast	yes	rep sanitised + coverage + tracer

Our row is last — no prior tool offers sound + fast + source-free + rep-movs sanitisation + coverage + tracing.

System model.



Threat model.

ATTACKER

Attacker

- Arbitrary input via stdin, network, file, argv
- No binary patching, no kernel access
- Assumes ASLR, NX, RELRO are on
- Goal: hijack control, leak data, DoS

DEFENDER

Defender

- Has only the stripped PIE ELF
- Adds sanitisation + coverage via RetroWrite
- Wants: find memory-safety bugs pre-deploy
- Constraint: near-native speed

OUT OF SCOPE

Out of scope

- Logic bugs, auth bypasses
- Side-channels and timing
- Non-PIE binaries (no relocations)
- Kernel or hypervisor compromise

Soundness holds only while PIE + intact relocations hold.

Three extensions.

E1

rep-movs / rep-stos ASan

Four shadow probes catch memcpy overflows that RetroWrite silently missed.

```
DISABLE_REP_FIX=1
```

E2

Basic-block coverage

AFL-compatible bitmap via raw mmap — no AFL runtime required.

```
DISABLE_COVERAGE=1
```

E3

Function-call tracer

Call-site trampolines log every entry to a circular buffer.

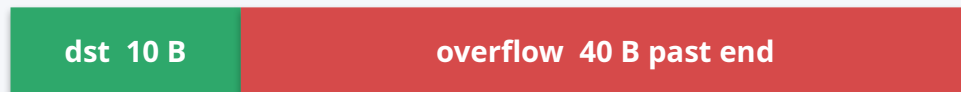
```
DISABLE_TRACE=1
```

Each pass drops in via `python3 retrowrite -m <mod>.`

E1 · Malloc safety, finally.

BEFORE

Silent memcpy overflow



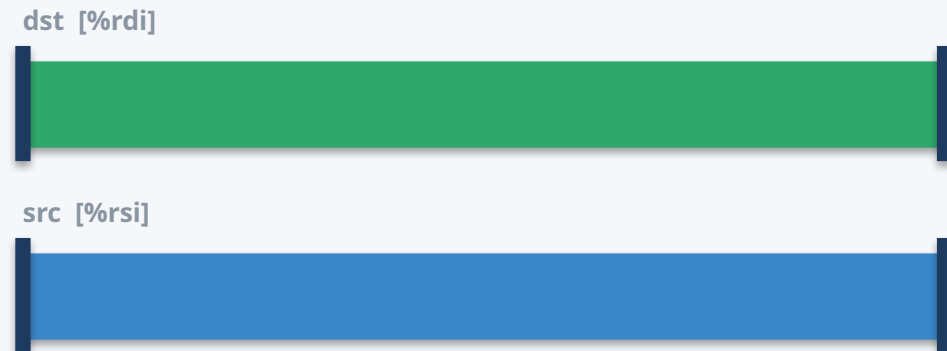
rep movsb copies 50 B into a 10 B buffer.

```
if mnemonic.startsWith("rep stos"):
    pass # XXX: TODO for more accurate check
# rep movs – not handled at all
```

Runtime: SEGV — silent corruption.

AFTER

Four boundary probes



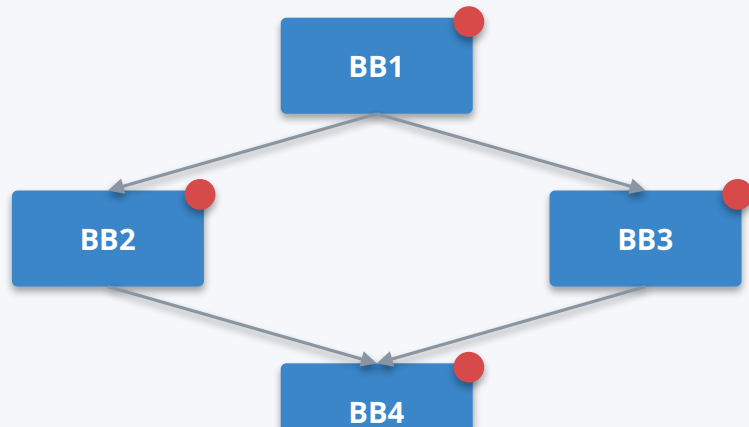
Each probe hits shadow memory and can fault safely.

Runtime: AddressSanitizer READ of size 1.

E2 and E3 · Coverage + tracing.

E2

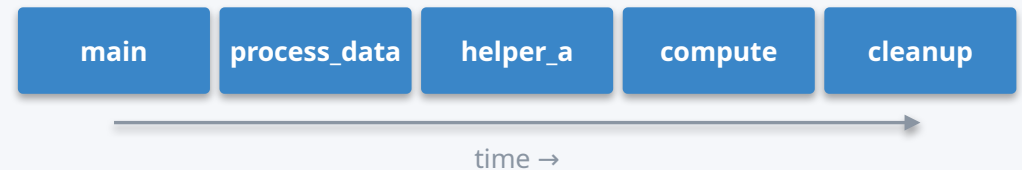
Basic-block coverage



Red mark = edge-hash trampoline at BB entry.
bitmap[cur ⊕ prev]++ · 64 KiB via raw mmap.

E3

Function-call tracer



```
subq $8, %rsp          ; 16-B re-align
pushq %rdi ... %r11     ; save 9 regs
leaq .LTRACE_NAME(%rip),%rdi
callq __trace_log_entry
```

Midterm results.

BUG DETECTION

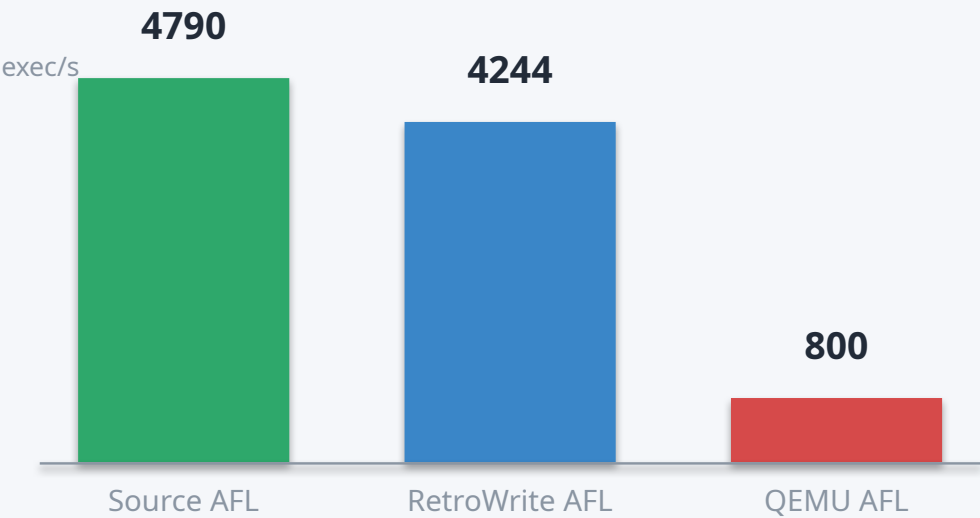
All four classes caught

Heap out-of-bounds	silent	caught
Use-after-free	silent	caught
Stack overflow	SEGV	caught
Double free	glibc abort	caught

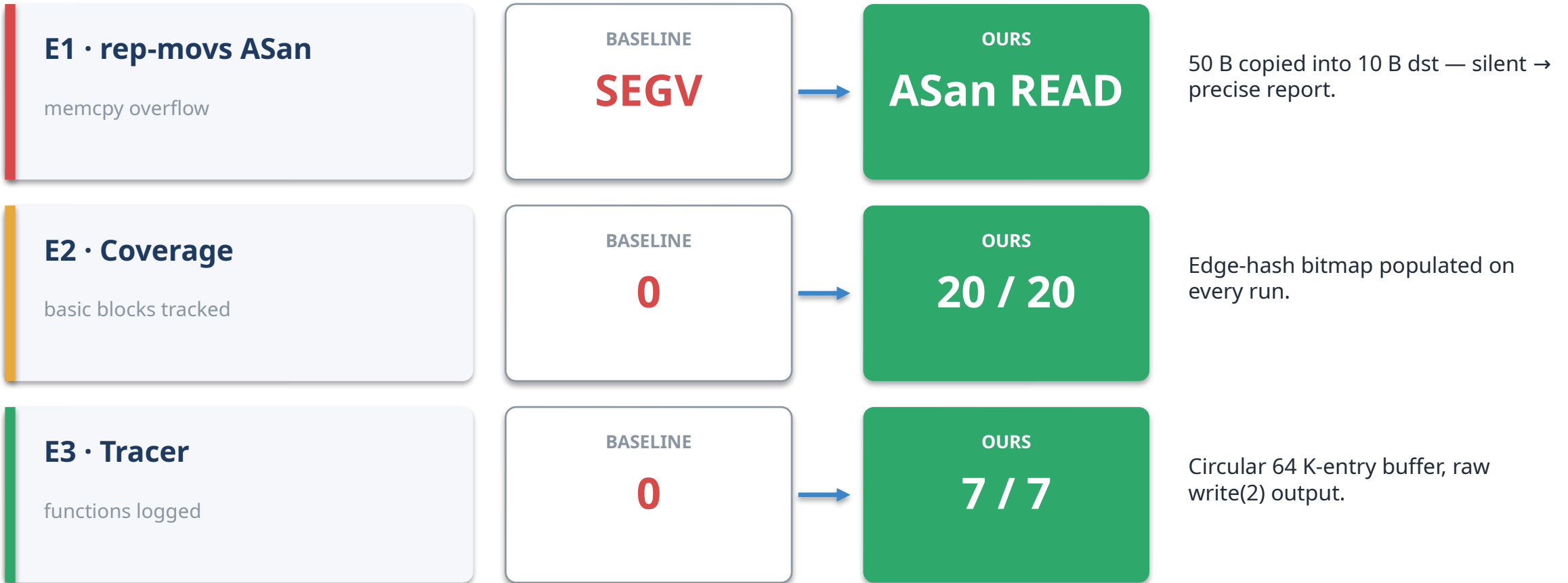
bzip2 round-trip: zero-byte diff.

AFL THROUGHPUT

88.6% of source speed



Endterm results.



Future work.



01

Non-PIE binaries

Probabilistic disassembly.



02

Stack canaries

Prologue / epilogue pass.



03

Per-object redzones

DWARF-aware boundaries.



04

Forward-edge CFI

Indirect-call ID matching.



05

Call-graph export

E3 → DOT at exit.



06

Trace-guided fuzzing

E2 bitmap $\oplus$ E3 call-stack.

Summary.

<600

lines of new Python

0

lines changed in librw_x64

3

new passes shipped

- **Reproduced the paper**
- **Closed a TODO in upstream**
- **Added coverage + tracer**

bzip2 round-trip · 88.6% of source · 5.3× faster than QEMU

rep movs / stos now instrumented — silent SEGV becomes an ASan report

x64 coverage in 240 LoC · tracer in 330 LoC · all toggleable

Relocation-guided rewriting is an extensible platform.

References · 1 / 2

- [1] S. Dinesh, N. Burow, D. Xu, M. Payer. RetroWrite: Statically Instrumenting COTS Binaries for Fuzzing and Sanitization. IEEE S&P 2020.
- [2] K. Serebryany, D. Bruening, A. Potapenko, D. Vyukov. AddressSanitizer: A Fast Address Sanity Checker. USENIX ATC 2012.
- [3] M. Zalewski. American Fuzzy Lop (AFL). 2017. lcamtuf.coredump.cx/afl/
- [4] F. Bellard. QEMU, a Fast and Portable Dynamic Translator. USENIX ATC 2005.
- [5] N. Nethercote, J. Seward. Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation. PLDI 2007.
- [6] C.-K. Luk et al. Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation. PLDI 2005.
- [7] D. Bruening, T. Garnett, S. Amarasinghe. An Infrastructure for Adaptive Dynamic Optimization. CGO 2003.
- [8] S. Wang, P. Wang, D. Wu. Reassembleable Disassembling. USENIX Security 2015.
- [9] R. Wang et al. Ramblr: Making Reassembly Great Again. NDSS 2017.
- [10] A. R. Bernat, B. P. Miller. Anywhere, Any-Time Binary Instrumentation. PASTE 2011.
- [11] G. J. Duck, X. Gao, A. Roychoudhury. Binary Rewriting without Control Flow Recovery. PLDI 2020.

References · 2 / 2

- [12] PaX Team. Address Space Layout Randomization (ASLR). 2003.
- [13] N. A. Quynh. Capstone: Next-Generation Disassembly Framework. Black Hat USA 2014.
- [14] MITRE. CWE-120: Buffer Copy without Checking Size of Input.
- [15] MITRE. CWE-121: Stack-Based Buffer Overflow.
- [16] S. Rostedt. ftrace: Function Tracer for Linux (kernel.org).
- [17] J. Weidendorfer. Callgrind: A Call-Graph Generating Cache Profiler. Valgrind.
- [18] J. Seward. bzip2 1.0.8. sourceware.org/bzip2, 2019.
- [19] K. Miller et al. Probabilistic Disassembly. ICSE 2019.
- [20] P. Chen, H. Chen. Angora: Efficient Fuzzing by Principled Search. IEEE S&P 2018.
- [21] Intel. Control-flow Enforcement Technology Specification, Rev. 3.0, 2019.
- [22] L. Szekeres, M. Payer, T. Wei, D. Song. SoK: Eternal War in Memory. IEEE S&P 2013.